



**UNIVERSITATEA
TEHNICĂ**
DIN CLUJ-NAPOCA

Bug Reporter

Assignement 3

Autori: Cimpan Alex-Ciprian, Ilco Doina-Cezara, Marcu Andrei
Grupa: 30233

FACULTATEA DE AUTOMATICA
SI CALCULATOARE

Mai 2026

Cuprins

1	Introducere	3
2	Tech stack	3
3	Arhitectura Sistemului	4
3.1	Diagrama de Arhitectura	4
3.2	Descrierea Arhitecturii	4
3.3	Flux de Comunicare	5
4	Diagrame	5
4.1	Diagrama Entitate-Relație	5
4.2	Diagrama Secvență - Flux Statistici	6
4.3	Diagrama Secvență - Flux Ban Utilizator	7
4.4	Diagrama de Clase	8
5	Frontend	9
5.1	Stack Tehnologic	9
5.2	Structură Proiect	9
5.3	Pagini Principale	10
5.3.1	Login și Register	10
5.3.2	Bugs	10
5.3.3	BugDetail	11
5.3.4	CreateBug și EditBug	11
5.3.5	Profile	11
5.3.6	ModeratorDashboard	11
6	Securitate	12
7	Pipeline CI/CD (GitHub Actions)	12
7.1	Workflow de Testare - <code>testing.yml</code>	12
7.2	Workflow de Linting - <code>lint.yml</code>	12
7.3	Reguli de Linting	12
7.3.1	Java - Checkstyle	12
7.3.2	React - ESLint	13
8	Testare	13
8.1	Teste Unitare Backend	13
8.2	Teste E2E - Cypress	13

9	Baza de Date	13
9.1	Configurare	13
9.2	Tabele generate automat	14
10	Implementare Bonus Features	14
10.1	Bonus Feature 1: Sistem de Scoring Utilizator	14
10.1.1	Reguli de Calcul	14
10.1.2	Formula Matematică	15
10.1.3	Arhitectură	15
10.2	Bonus Feature 2: Sistem de Privilegii Moderator	15
10.2.1	Roluri de Utilizator	15
10.2.2	Privilegiile Moderatorului	16
10.2.3	Implementare	16

1 Introducere

BugReporter este o aplicatie web care permite utilizatorilor sa raporteze, sa urmareasca si sa gestioneze bug-uri software. Utilizatorii pot crea rapoarte de bug-uri cu tag-uri, pot comenta bug-urile existente si pot efectua operatii CRUD complete pe toate entitatile. Aplicatia include un sistem de votare (upvote/downvote) pe bug-uri si comentarii, cu calcul automat al scorului de reputatie al utilizatorilor. Assignment 3 extinde aplicatia BugReporter cu o arhitectura CI/CD robusta, microservicii independente si functionalitati avansate de moderare. Bonus features-urile adauga capabilitate de gestionare a utilizatorilor si un sistem de notificari.

2 Tech stack

Componenta	Tehnologie	Versiune
Backend principal	Spring Boot	4.0.3
Microserviciu Statistici	Spring Boot + JDBC	4.0.3
Microserviciu Notificari	Spring Boot + Mail	4.0.3
Frontend	React	19.2.4
Build tool frontend	Vite	5.4.0
Baza de date	PostgreSQL	16
ORM	Hibernate / JPA	–
Autentificare	BCrypt (Spring Security)	–
Testare backend	JUnit 5 + Mockito	–
Testare E2E	Cypress	–
Linting Java	Maven Checkstyle Plugin	3.3.1
Linting React	ESLint	9.x
CI/CD	GitHub Actions	–
Comunicare servicii	RestTemplate (HTTP)	–

Tabela 1: Stack tehnologic

3 Arhitectura Sistemului

3.1 Diagrama de Arhitectura

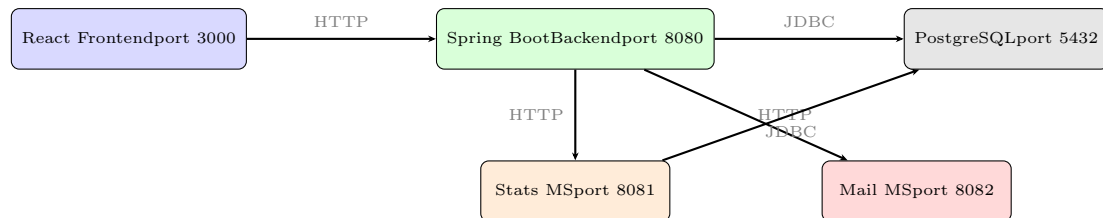


Figura 1: Arhitectura generala a aplicatiei

3.2 Descrierea Arhitecturii

Sistemul BugReporter este construit pe o arhitectura distribuita cu urmatoarele componente:

React Frontend (port 3000)

Interfata utilizator construita cu React 19. Comunicare exclusiva cu backend-ul principal prin HTTP/REST API.

Spring Boot Backend (port 8080)

Nucleul aplicatiei care gestioneaza CRUD-uri, voturi, autentificare, autorizare si proxuiera cererilor catre microservicii. Comunica cu baza de date prin JPA/Hibernate.

Microserviciu Statistici (port 8081)

Serviciu independent care calculeaza statistici agregate (total buguri, comentarii, utilizatori per status). Foloseste JdbcTemplate pentru interogari SQL directe.

Microserviciu Notificari (port 8082)

Serviciu dedicat trimiterii notificarilor prin email (ban, stergere continut). Backend-ul nu este blocat de operatiuni de email.

PostgreSQL Database (port 5432)

Baza de date relationala unica, partajata intre toate componentele. Tabele: users, bug, comment, bug_vote, comment_vote, tag.

3.3 Flux de Comunicare

1. Frontend trimite HTTP requests catre Backend (port 8080)
2. Backend executa interogari la Database via JPA
3. Backend apeleaza Statistici MS (port 8081) via HTTP pentru date agregate
4. Backend apeleaza Mail MS (port 8082) via HTTP pentru notificari
5. Ambele microservicii acceseaza Database pentru citire/configurare

4 Diagrame

Această secțiune prezintă diagramele sistemului BugReporter, ilustrând arhitectura, structura datelor și fluxurile principale de comunicare între componente.

4.1 Diagrama Entitate-Relație

Diagrama prezintă cele șase entități principale ale sistemului. **User** poate raporta buguri, adăuga comentarii și vota. Fiecare utilizator are un rol (**USER** sau **MODERATOR**) și un câmp **isBanned** pentru gestionarea accesului. **Bug** are un status (**RECEIVED**, **IN_PROGRESS**, **SOLVED**) care se actualizează automat în funcție de activitatea din sistem. Entitățile **BugVote** și **CommentVote** rețin voturile utilizatorilor, pe baza cărora se calculează scorul fiecărui element.

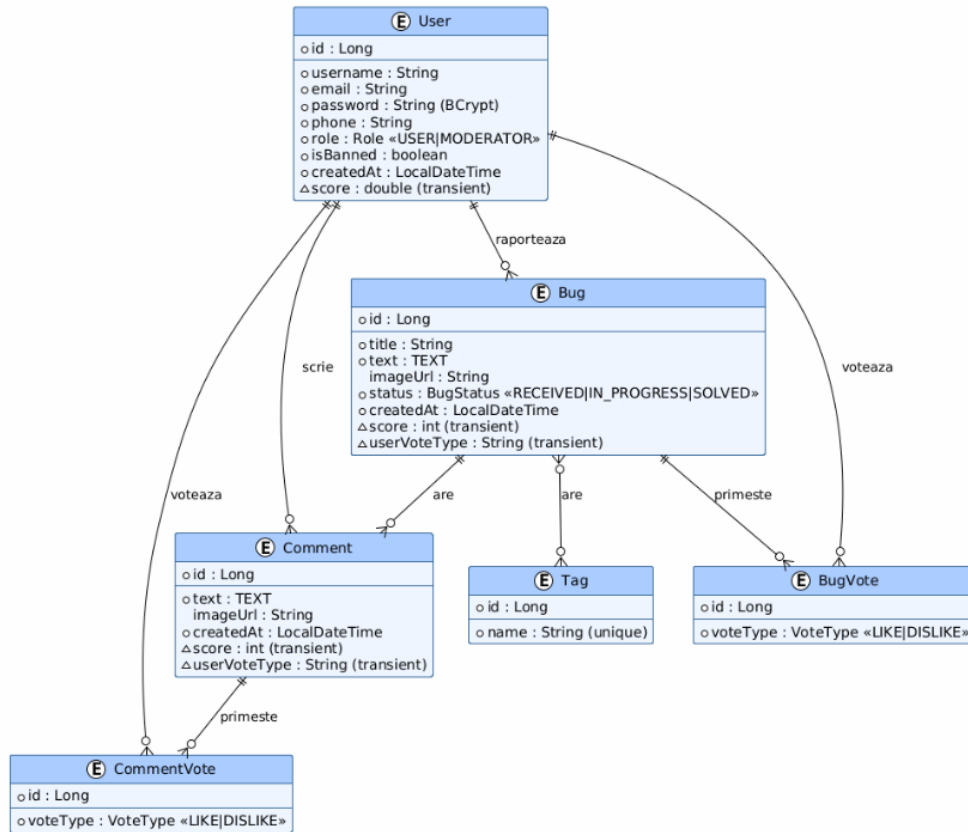


Figura 2: Diagrama Entitate-Relație

4.2 Diagrama Secvență - Flux Statistici

Diagrama ilustrează comunicarea dintre cele trei niveluri ale arhitecturii la accesarea paginii principale. Frontend-ul apelează `GET /api/stats` pe backend-ul principal, care la rândul său delegă cererea către microserviciul de statistici prin `RestTemplate`. Microserviciul execută interogări SQL directe prin `JdbcTemplate` și returnează un obiect `StatsResponse` cu totalul de buguri, comentarii și utilizatori, grupat și pe statusuri.

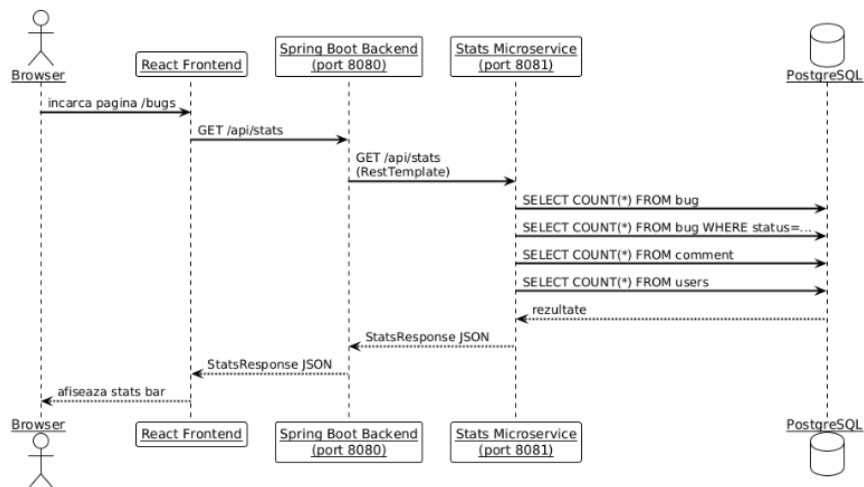


Figura 3: Flux cerere statistici prin microserviciu

4.3 Diagrama Secvență - Flux Ban Utilizator

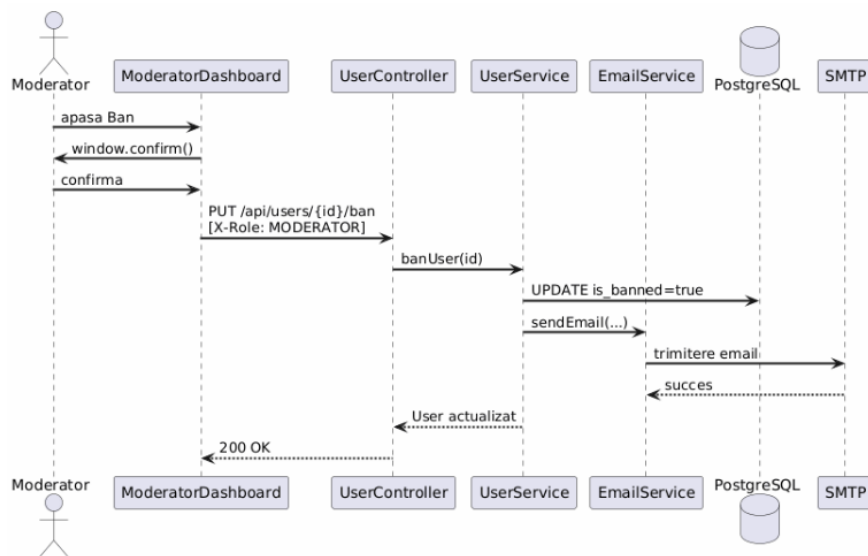


Figura 4: Flux ban utilizator de către moderator

Diagrama descrie fluxul complet al acțiunii de ban inițiată de un moderator din *Moderator Dashboard*. Rolul utilizatorului curent este transmis prin headerul HTTP `X-Current-User-Role` și verificat în controller înainte de a apela service-ul. După setarea câmpului `isBanned` în baza de date, sistemul trimite automat un email de notificare utilizatorului banat prin `EmailService`. Interfața se actu-

alizează imediat, afișând badge-ul *Banned* în tabel.

4.4 Diagrama de Clase

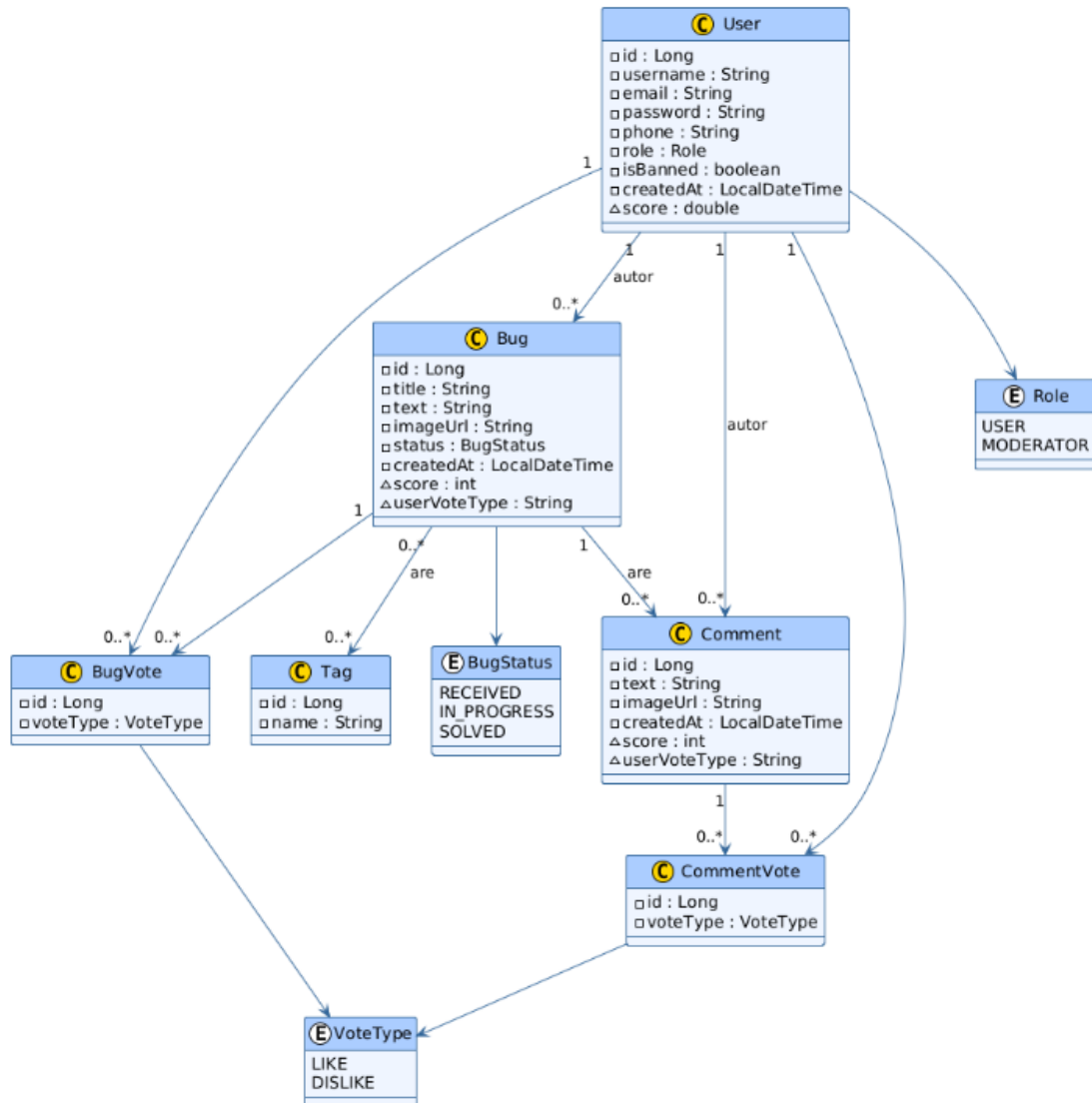


Figura 5: Diagrama1 de clase a sistemului

Diagrama prezintă structura internă a aplicației organizată pe trei niveluri: **controllere**, **servicii** și **entități**. Controllerele primesc cererile HTTP și le delegă serviciilor corespunzătoare. Serviciile conțin logica de business și accesează baza de date prin repository-uri JPA. Entitățile mapează tabelele din PostgreSQL

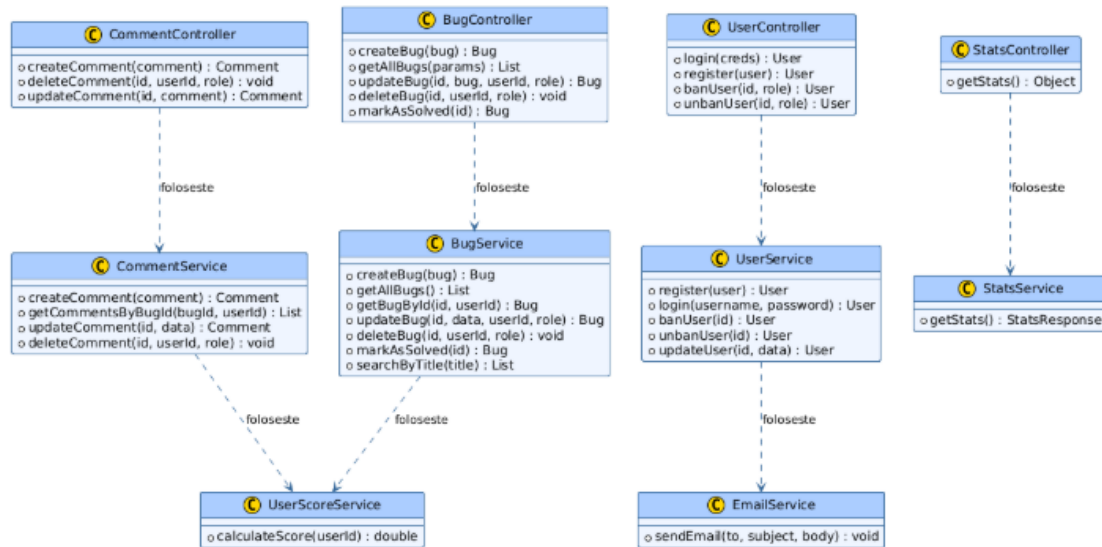


Figura 6: Diagrama2 de clase a sistemului

și includ câmpuri `@Transient` pentru date calculate la runtime (scor, tipul de vot al utilizatorului curent) care nu sunt persistate. Relațiile dintre entități reflectă constrângerile din schema bazei de date.

5 Frontend

5.1 Stack Tehnologic

- **Framework:** React 19.2.4
- **Build Tool:** Vite 5.4.0
- **Routing:** React Router DOM 7.13.2
- **Linting:** ESLint 9.0.0
- **Testing:** Cypress 15.15.0
- **Language:** JavaScript (ESM modules)

5.2 Structură Proiect

src/

```

App.jsx          # Root component cu routing
main.jsx         # Entry point
api.js           # API calls către backend
style.css        # Global styles
components/
  Navbar/        # Navigation bar
pages/
  Login/         # Authentication page
  Register/      # User registration
  Bugs/          # Lista buguri
  BugDetail/     # Detalii bug
  CreateBug/     # Creare bug nou
  EditBug/       # Editare bug
  Profile/       # Profil utilizator
  ModeratorDashboard/# Panou moderator

```

5.3 Pagini Principale

5.3.1 Login și Register

Pagini de autentificare ce permit:

- **Login:** Accesul cu username și parolă
- **Register:** Crearea cont nou cu email și telefon
- Stocarea utilizatorului în localStorage
- Redirectare automată la /login dacă nu autentificat

5.3.2 Bugs

Pagina principală cu:

- Listarea buguri cu filtru, căutare, sortare
- Buton pentru creare bug nou
- Card-uri cu informații bug (titlu, autor, status)
- Accesul la detalii prin click pe card

5.3.3 BugDetail

Vizualizare detaliată a unui bug:

- Titlu, descriere, status, prioritate
- Vot pozitiv/negativ pe bug
- Comentarii cu posibilitate de răspuns
- Vot pe comentarii
- Opțiuni editare/ștergere dacă autorul

5.3.4 CreateBug și EditBug

Formulare pentru gestionarea buguri:

- Titlu, descriere, fișiere atașate
- Selectare status (OPEN, IN_PROGRESS, FIXED)
- Selectare prioritate (LOW, MEDIUM, HIGH)
- Validare și trimitere la backend

5.3.5 Profile

Pagina profil utilizator:

- Afișare date utilizator (username, email, telefon)
- Scor utilizator (din Bonus Feature 1)
- Listarea buguri și comentarii proprii
- Logout

5.3.6 ModeratorDashboard

Panou accesibil doar moderatorilor:

- Listarea utilizatori
- Butoane pentru ban/unban
- Ștergere buguri inadequate
- Ștergere comentarii
- Vizualizare rapoarte

6 Securitate

- Autentificare pe bază de `localStorage`
- Verificare rol pentru acces rute moderator
- Headers custom în API calls pentru context utilizator
- Protecție rute: nedatată fără user → redirect `/login`

7 Pipeline CI/CD (GitHub Actions)

7.1 Workflow de Testare - `testing.yml`

Declansare: la fiecare `push` sau `pull_request` pe branch-ul `main`.

Job: `backend-tests` Porneste un serviciu PostgreSQL (imagine `postgres:16`), compileaza proiectul Maven si ruleaza toate testele JUnit cu `mvn test`.

Job: `cypress-tests` Instaleaza dependintele Node.js si ruleaza testele E2E Cypress.

7.2 Workflow de Linting - `lint.yml`

Declansare: la fiecare `push` sau `pull_request` pe branch-ul `main`.

Job: `checkstyle` Foloseste JDK 17 (Temurin) si ruleaza `mvn checkstyle:check` cu un fisier de reguli personalizat (`checkstyle.xml`).

Job: `eslint` Foloseste Node.js 20 si ruleaza `npm run lint` cu ESLint 9 in configuratie flat config.

7.3 Reguli de Linting

7.3.1 Java - Checkstyle

- Interzicerea tab-urilor (`FileTabCharacter`)
- Import-uri neutilizate interzise (`UnusedImports`)
- Star imports interzise (`AvoidStarImport`)
- Conventii de denumire: clase, metode, variabile, parametri (`TypeName`, `MethodName` etc.)
- Lungime maxima linie: 200 caractere

7.3.2 React - ESLint

- Variabile neutilizate: avertisment (`no-unused-vars`)
- Variabile nedecarate: eroare (`no-undef`)
- Variabile JSX marcate ca utilizate (`react/jsx-uses-vars`)

8 Testare

8.1 Teste Unitare Backend

Acopera serviciile principale cu JUnit 5 + Mockito:

- `UserServiceTest`: ban, unban, inregistrare
- `BugServiceTest`: creare, stergere, filtrare
- `CommentServiceTest`: creare, blocare pe SOLVED, delete cu permisiuni
- `BugVoteServiceTest`: votare, schimbare vot

8.2 Teste E2E - Cypress

- Autentificare (login/register)
- Creare si vizualizare bug
- Editare si stergere bug
- Adaugare comentarii

9 Baza de Date

9.1 Configurare

Aplicația folosește două baze de date PostgreSQL:

- `bugtracker` – baza principală, folosită în dezvoltare și producție
- `bugtracker_test` – baza de test, cu `ddl-auto=create` (tabelele se recreează la fiecare rulare a testelor)

9.2 Tabele generate automat

Hibernate generează automat următoarele tabele pe baza entităților JPA:

- **users** – utilizatori (id, username, email, password, created_at)
- **bug** – bug-uri (id, title, text, image_url, status, created_at, author_id)
- **comment** – comentarii (id, text, image_url, created_at, author_id, bug_id)
- **tag** – tag-uri (id, name)
- **bug_tags** – tabela de legătură ManyToMany între bug și tag (bug_id, tag_id)
- **bug_votes** — voturi pe bug-uri (id, user_id, bug_id, vote_type), constrangere unica (user_id, bug_id)
- **comment_votes** — voturi pe comentarii (id, user_id, comment_id, vote_type), constrangere unica (user_id, comment_id)

10 Implementare Bonus Features

10.1 Bonus Feature 1: Sistem de Scoring Utilizator

Sistemul de scoring permite calcularea unei note pentru fiecare utilizator pe baza activităților sale pe platformă: voturile primite pe buguri și comentarii, precum și voturile negative date de utilizator pe comentariile altor utilizatori.

10.1.1 Reguli de Calcul

Scorul utilizatorului se calculează conform următoarelor reguli:

- **Bug cu vot pozitiv: +2.5 puncte** per vot
- **Bug cu vot negativ: -1.5 puncte** per vot
- **Comentariu cu vot pozitiv: +5 puncte** per vot
- **Comentariu cu vot negativ: -2.5 puncte** per vot
- **Vot negativ dat pe comentariile altor utilizatori: -1.5 puncte** per vot

Scorul poate fi negativ dacă utilizatorul primește mai multe voturi negative decât pozitive.

10.1.2 Formula Matematică

Formula de calcul a scorului este:

$$\text{Score} = (B_L \times 2.5) - (B_D \times 1.5) + (C_L \times 5.0) - (C_D \times 2.5) - (C_{D,other} \times 1.5)$$

unde:

- B_L = numărul de voturi pozitive pe bugurile utilizatorului
- B_D = numărul de voturi negative pe bugurile utilizatorului
- C_L = numărul de voturi pozitive pe comentariile utilizatorului
- C_D = numărul de voturi negative pe comentariile utilizatorului
- $C_{D,other}$ = numărul de voturi negative date de utilizator pe comentariile altor utilizatori

10.1.3 Arhitectură

Sistemul de scoring este implementat ca serviciu Spring (UserService) care accesează datele de voturi din baza de date și aplică formula matematică descrisă anterior. Serviciul expune metoda `calculateScore(userId)` care returnează scorul utilizatorului ca valoare double.

10.2 Bonus Feature 2: Sistem de Privilegii Moderator

Sistemul de privilegii moderator permite unor utilizatori selectați (cu rolul de MODERATOR) să exercite funcții administrative pe platformă pentru a menține calitatea conținutului și a aplica regulile comunității.

10.2.1 Roluri de Utilizator

```
1 package com.bugtracker.entity;  
2  
3 public enum Role {  
4     USER,           // Utilizator standard  
5     MODERATOR       // Moderator cu drepturi speciale  
6 }
```

Listing 1: Role.java

10.2.2 Privilegiile Moderatorului

Moderatorii au următoarele privilegii și responsabilități:

1. Gestionarea Bugurilor și Comentariilor

- **Ștergerea bugurilor** - pot șterge buguri care nu respectă regulile comunității
- **Editarea bugurilor sau comentariilor** - pot corecta conținutul nepotrivit
- **Blocarea utilizatorilor** - pot bloca temporar utilizatori care au comportament inadecvat

2. Acțiuni Disponibile

- Ștergerea unui bug sau comentariu dacă sunt inadecvate
- Editarea unei pagini de bug sau comentariu
- Blocarea unui utilizator pentru o perioadă nedeterminată în caz de comportament inadecvat
- Deblocarea unui utilizator care a fost blocat anterior
- Trimiterea unui mesaj de e-mail și SMS către utilizatorul blocat

3. Notificări

Utilizatorii blocați sunt notificați prin:

- **E-mail** - mesaj detaliat cu motivul blocării
- **SMS/WhatsApp** - alert imediat la momentul blocării

10.2.3 Implementare

Sistemul de moderație este implementat prin:

- Enum `Role` cu valorile `USER` și `MODERATOR`
- Campo boolean `banned` în entitatea `User`
- Serviciu `UserService` cu metode `banUser()` și `unbanUser()`
- Integrare `EmailService` și notificări SMS via `WhatsApp API`

Procesul de suspendare este tranzacțional și asigură că utilizatorul este marcat ca suspendat și notificat prin email și SMS.